

EECS 498 • AASE • FALL 2026

From pair-coding to agent orchestration

LECTURE 06 • SEP 17, 2026

Where the build starts

- The packet: seven features, every one numbered
- `design/SYSTEM.md` and three diagrams before code
- Then increment specs, one at a time

The packet lands after this lecture. Specs before code is graded.

Tuesday's promise: today the loop stops being **someone else's**.

The seam

WHERE APPLY STOPS AND ANALYZE STARTS

Three words for Aider's loop



Fixed

A small set of cycles, chosen when it was written.



Shallow

A turn never compounds on itself.



Human-framed

You choose files, each step, and when it ends.

The three words as requirements

In your spec	Which word
§1 — no discovery, no commands, no tool choice, no self-repair	Fixed
F1.4, F5.1 — one request, one reply, done	Shallow
F2.1, F6.6 — you /add , you approve	Human-framed

F6.5, F6.6, F6.7

An approval layer for files

- A complete unified diff before any write
- Explicit approval, or nothing happens
- Blank, EOF, interrupt: all mean no

Aider writes first. Yours must ask. One kind of action, and the most important code you write this month.

EVERY TURN, FOR TWO WEEKS

Three decisions a human supplies

1. Which files are in play
2. What the next step is
3. When the turn is over

Take the second one away.

Deepen and generalize

TWO MOVES, AND THEY LAND IN DIFFERENT WEEKS

A tool layer with many verbs

- `read_file` — the program chooses, not you
- `write_file` — same edits, no `/add`
- `run_command` — the one that changes your machine

The shape of a tool call

```
{"tool_calls": [  
  {"name": "read_file", "arguments": {"path": "cli.py"}},  
  {"name": "write_file", "arguments": {"path": "cli.py", "content": "..."}},  
  {"name": "run_command", "arguments": {"command": "pytest -q tests/"}}  
]}
```

Three verbs, one shape. A parseable request, not prose. Your code decides what happens next.

The model never runs anything. It
asks.

The loop that picks its step

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text          # the model is done
    for call in reply.tool_calls:
        result = execute(call)     # your code, your rules
        messages.append(result)
```

WHAT DID NOT CHANGE

What the harness still does

	Pair-programmer (M1)	Agent (M2)
Emits text, nothing else	the model	the model
Reads, writes, runs commands	your code	your code
Picks the next action	you	the model

Two rows did not move. The third row is the whole change.

Four stop conditions

- A cap on iterations
- A cap on tokens
- A detector for repeated identical calls
- A clean interrupt, mid-loop

Handing one task over

Pick one task from `task1`, the project you drove Aider through for two weeks, that you would hand over completely, so that you type it once and read the result at the end.

What would the program be allowed to do without asking you first?

What is the earliest thing on that list you would refuse?

Which was harder to write down, the permission or the refusal?

Agent v0

WEEKS 5 TO 7, IN THE REPOSITORY YOU ALREADY HAVE

ONE REPOSITORY, WEEKS 1 TO 15

From pair-programmer to v0

- No new repo, no fresh start
- The same git history, all term
- It ends as the assistant you demo

What each week delivers

Week	The piece	The question it answers
5	Tools, the loop, stop conditions, manual approval	Why does it not stop?
6	Auto-approval rules, decision log, endpoint measured	What did the human silently supply?
7	Eval harness and the gate	Where does it fail, measured?

BY DESIGN, NOT BY ACCIDENT

What v0 will not do

- No command sandbox
- No session across runs
- No sub-agents, no MCP, no hooks
- No compaction when the context fills

That list is the **syllabus** for
Create.

The approval layer

FORCED BY A MECHANISM, NOT CHOSEN OUT OF CAUTION

TODAY

What git covers

- One commit per accepted, nonempty edit set
- `/undo` walks back your session's commit
- Refuses when a human commit intervenes

A complete guarantee, while files are all you touch.

The hole a shell opens

```
/undo → rm -rf build/  
/undo → git reset --hard  
/undo → curl ... | sh  
/undo → pip install anything
```

None of these were ever in the repository.

Four steps of an approval layer

1. **Intercept** before execution, never after
2. **Prompt** to allow or deny, command shown
3. **Consult** a rules file for auto-approval
4. **Log** the decision, whatever it was

THE ORDER IS THE LESSON

Manual **first**, rules second

- Week 5: the prompt, about ten lines
- Week 6: the rules file, after a week of watching

A rule written before you have seen the traffic is a guess.

The tension the gate measures

- **Too tight** — the run stalls, no human to ask
- **Too loose** — you find out the other way

Your approval log is graded evidence.

The weeks ahead

When	What
Mon 21 / Tue 22	Lab02: a 40-minute actions example, then build time
Tue Sep 22	L07, where Analyze starts
Thu Sep 24, evening	Hackathon 1, on a supplied application
Tue Oct 6, 11:59 PM	The build is due, then the Analyze packet

Nothing about Analyze is handed out next week. Spend Lab02's second half on the build.

TAKE THESE WITH YOU

Three questions

1. Which decision would you give up first, and what breaks?
2. What would you allowlist, and what would you never?
3. Forty steps, wrong answer: what did you log?

Tuesday, we **open** the client.

Questions?